Evaluation of MLIR for Intermediate Representation in a Fault-Tolerant Quantum Compiler

1. Introduction

Intermediate Representations (IRs) play a central role in compiler design, acting as an abstraction layer between high-level program descriptions and low-level hardware execution. In the context of quantum computing—particularly for **fault-tolerant quantum compiler design**—the IR must support circuit transformations, hardware-aware optimizations, and scalable representation of quantum operations [7].

At the beginning of this work, **Multi-Level Intermediate Representation (MLIR)** was considered as a promising framework for representing the IR of a fault-tolerant quantum compiler. MLIR, developed within the LLVM ecosystem, provides an extensible infrastructure for defining domain-specific dialects and transformation passes [1]. Several recent research works suggested that MLIR could be a strong candidate for representing quantum circuits and enabling compiler optimizations [2][3].

However, after an extensive exploration and comparative study of existing quantum software ecosystems, we decided **not to proceed with MLIR as the primary IR representation**. Instead, we shifted towards using established quantum representations and toolchains, specifically:

- **pytket**
- **Qiskit-based ecosystems**

This report documents:

- The initial motivation for exploring MLIR
- The experiments and investigations conducted using MLIR
- The comparative analysis performed with existing tools
- The reasons for selecting **pytket and Qiskit instead of MLIR**

2. Background

2.1 Intermediate Representation in Quantum Compilers

An IR in a quantum compiler serves multiple purposes:

- Representation of quantum circuits
- Enabling transformations and optimizations
- Hardware-specific mapping
- Fault-tolerance analysis and compilation
- Translation between quantum languages and hardware instructions

Unlike classical compilation, quantum IR must account for:

- Quantum gate semantics
- Circuit depth optimization

- Qubit connectivity constraints
- Error correction overhead
- Hardware calibration parameters

Therefore, selecting the appropriate IR framework is a key design decision in the compiler architecture. Modern quantum compilers often rely on **domain-specific circuit representations** rather than general-purpose compiler IRs [7].

---

3. Initial Motivation for Using MLIR

3.1 Overview of MLIR

MLIR is a compiler infrastructure designed to support multiple levels of abstraction in program representation. It allows developers to define **custom dialects**, enabling domain-specific operations and transformations [1].

Key features include:

- Extensible dialect system
- Structured transformation passes
- Integration with LLVM infrastructure
- Support for multiple abstraction layers

These capabilities made MLIR an appealing option for representing **multi-level quantum compilation pipelines**.

3.2 Why MLIR Appeared Suitable for Quantum IR

Several aspects of MLIR initially suggested that it could be well suited for quantum compiler development.

1. Multi-level Abstraction

Quantum compilation often involves several abstraction layers:

- Algorithm level
- Logical circuit level
- Fault-tolerant logical gate level
- Physical hardware instructions

MLIR's design allows IRs to exist at multiple abstraction levels simultaneously [1].

---

2. Custom Dialects

MLIR allows defining **custom dialects**, which could represent:

- Quantum gates
- Stabilizer operations
- Error correction primitives

- Hardware-specific instructions

This flexibility made it attractive for representing fault-tolerant compilation workflows.

Several recent projects have explored **MLIR-based quantum dialects**, demonstrating how quantum circuits could be represented within the MLIR ecosystem [2][3].

---

3. Transformation Infrastructure

MLIR includes a well-developed pass infrastructure that could theoretically support:

- Circuit optimization
- Gate decomposition
- Error-correction transformations
- Hardware mapping

---

4. Research Interest

Several recent research papers suggested MLIR-based approaches for quantum compilers, proposing frameworks where quantum circuits are represented using MLIR dialects [2][3].

These works motivated the initial investigation of MLIR as a potential IR backend.

---

4. Exploration and Experiments with MLIR

During the investigation phase, we explored the feasibility of using MLIR as the primary IR for the compiler.

4.1 Understanding the MLIR Architecture

We studied the MLIR ecosystem to understand:

- Dialect creation
- Operation definitions
- Pass pipelines
- IR serialization formats

This involved exploring MLIR documentation and research papers describing quantum compiler infrastructures built on MLIR [1][2].

4.2 Designing a Potential Quantum Dialect

A conceptual design was explored where a **quantum dialect** would include operations such as:

- Quantum gate operations (X, H, CNOT, etc.)
- Measurement operations
- Qubit allocation

- Circuit transformations

However, it quickly became clear that implementing a fully functional dialect would require significant development work.

4.3 Evaluating Integration with Quantum Toolchains

Another challenge was integration with existing quantum ecosystems. MLIR does not natively support many standard quantum formats such as:

- OpenQASM
- Circuit representations used by quantum SDKs
- Hardware calibration metadata

Supporting these would require implementing additional infrastructure.

---

5. Benchmark-Based Evaluation of Existing Quantum Compiler Frameworks

The link to the report

In addition to the architectural evaluation of MLIR, a practical benchmarking study was conducted to compare the performance of existing quantum compiler frameworks.

The benchmark compared two widely used quantum compilation frameworks:

- **Qiskit (IBM)** [4]
- **pytket (Quantinuum)** [5]

Both frameworks were used to compile and execute the same quantum circuits on **IBM's real quantum hardware backend (`ibm_torino`)**.

The benchmark experiment consisted of the following circuit structure:

1. Apply **X gate to all qubits**
2. Apply **Quantum Fourier Transform (QFT)**
3. Apply **Inverse Quantum Fourier Transform (IQFT)**
4. Measure all qubits

The expected output was a deterministic **all-ones bitstring**.

The benchmark measured multiple metrics including:

- Circuit depth after routing
- Two-qubit gate count
- Compilation time
- Execution time
- Memory consumption during compilation

- Fidelity of the output distribution

These metrics allow evaluation of both **compiler efficiency and hardware execution quality**.

---

6. Benchmark Results

The compiled circuits from both frameworks were executed on IBM's `ibm_torino` quantum device.

Circuit Metrics After Routing

Framework	Circuit Depth	Two-Qubit Gates
Qiskit	352	106
pytket	250	78

The pytket-compiled circuit produced a **shallower circuit with fewer two-qubit gates**, which are the most error-prone operations in quantum hardware.

Runtime Metrics

Framework	Compile Time (s)	Execution Time (s)
Qiskit	2.01	8.33
pytket	0.769	7.36

pytket showed **significantly faster compilation time** compared to Qiskit.

Memory Usage

Framework	Peak Memory (MiB)
Qiskit	7.12
pytket	0.656

The pytket compilation process used **an order of magnitude less memory** than Qiskit.

Hardware Result Quality

Framework	Target State Probability	Hellinger Fidelity
Qiskit	0.1719	0.2349
pytket	0.4668	0.4372

pytket produced **substantially higher fidelity results** and better probability of measuring the expected output state.

These results demonstrate that **compiler optimizations significantly affect real hardware performance**.

---

7. Implications for IR Design

The benchmark results highlight an important observation: modern quantum compiler frameworks already provide **highly optimized intermediate representations and transformation pipelines**.

For example:

Qiskit IR Stack

```
QuantumCircuit
      ↓
DAGCircuit (compiler IR)
      ↓
Basis Gate Circuit
      ↓
OpenQASM
      ↓
Hardware execution
```

Qiskit internally represents circuits using a **Directed Acyclic Graph representation (DAGCircuit)** that enables dependency tracking and circuit optimization during compilation [4].

---

pytket IR

pytket internally represents circuits using a **graph-based circuit representation**, enabling efficient:

- gate rewriting
- circuit simplification
- routing
- hardware mapping

The pytket compiler architecture is designed to be **retargetable across different quantum hardware platforms** [5].

---

8. Reconsidering MLIR

Initially, MLIR was considered because of its ability to support **multi-level compiler representations and domain-specific dialects** [1].

However, the benchmarking and ecosystem exploration revealed several practical challenges.

8.1 Lack of Quantum Ecosystem Integration

Unlike pytket and Qiskit, MLIR does not yet have a mature ecosystem for:

- quantum circuit representations
- hardware routing algorithms
- quantum-specific optimization passes

8.2 Infrastructure Development Overhead

Adopting MLIR would require implementing large parts of the quantum compiler stack from scratch, including:

- a custom **quantum dialect**
- circuit transformation passes
- routing algorithms
- hardware mapping layers
- conversion pipelines to existing quantum formats

This would effectively mean **rebuilding infrastructure that already exists in established quantum frameworks**.

8.3 Lack of Hardware Integration

Current quantum frameworks such as Qiskit and pytket already integrate directly with quantum hardware providers. For example, the benchmark experiments executed circuits directly on **IBM's `ibm_torino` quantum device**.

MLIR does not currently provide native integration with such hardware ecosystems.

---

9. Final Decision: Moving Away from MLIR

Based on both **architectural analysis and empirical benchmarking**, the project decided not to adopt MLIR as the primary intermediate representation.

The main reasons were:

1. **Existing frameworks already provide optimized IRs**
2. **pytket demonstrated strong performance in real hardware benchmarks**
3. **OpenQASM provides a widely supported standardized circuit format** [6]
4. **MLIR would require substantial reimplementation effort**

Therefore, the project shifted towards leveraging **pytket and OpenQASM-based representations**, which provide mature infrastructure for quantum circuit compilation.

This decision allows development efforts to focus on **fault-tolerant compilation techniques and algorithm-level optimizations**, rather than building a new IR framework.

References

[1] Lattner, C. et al. **MLIR: A Compiler Infrastructure for the End of Moore's Law.** CGO 2021.

[2] Moses, S. A. et al. **A Compiler Infrastructure for Quantum Computing.** arXiv:2001.02873.

[3] Nguyen, D. et al. **Towards an MLIR-based Quantum Compiler Infrastructure.** arXiv preprint.

[4] Aleksandrowicz, G. et al. **Qiskit: An Open-source Framework for Quantum Computing.** Zenodo, 2019.

[5] Sivarajah, S. et al. **t|ket⟩: A Retargetable Compiler for NISQ Devices.** Quantum Science and Technology, 2020.

[6] Cross, A. W. et al. **Open Quantum Assembly Language.** arXiv:1707.03429.

[7] Bharti, K. et al. **Noisy Intermediate-Scale Quantum Algorithms.** Reviews of Modern Physics, 2022.